

A view of the landscape so far. Regular languages:

- Model for computation: DFAs.
- Syntactic characterization: Regular expressions.

Which would you prefer for each of the following:

- Test membership of 20 strings.
- Generate 20 shortest strings.

Are DFAs computers? Pumping lemma $\Rightarrow$ No!

Problem: Finite memory! Can't even recognize $\{a^n b^n \mid n \geq 0\}$.

Next few weeks:

- A model of computation with infinite memory, limited access.
- A syntactic characterization.

Pushdown Automata

Idea: NFAs with a stack.

NFAs:

- Read a character.
- Transition to a new state.

Pushdown automata:

- Read a character.
- Pop something off stack. (What we pop affect the next two steps.)
- Transition to a new state.
- Push something(s) (or nothing) onto stack.

Stack: Can store infinite information, but can't access old information easily.

Observations:

- Can accept every regular language.
- Will also be able to handle $\{a^n b^n \mid n \geq 0\}$ (keep count on stack)

Context-Free Grammars

Idea: Syntactical description of programming languages.

Show how to make an if statement.

Here's another "programming language" $\langle stmt \rangle := a\langle stmt \rangle b \mid \epsilon$

Apply once.

Note: Not allowed to have things in angle braces in the final statement.

Definition: A context-free grammar (CFG) is a quadruple $G = (N, \Sigma, P, S)$.

- N - set of *nonterminals*. Above: $\langle stmt \rangle$. (What about in earlier example?)
Usually: Capital letters.
- Σ - alphabet of terminal characters.
For PL example:
For above example:
Usually: Lowercase letters.
- P - *Productions* Sets of pairs: $N \times (N \cup \Sigma)^*$

Shorthand: vertical bars.

- $S \in N$: Start symbol.

Def. Let $\alpha, \beta \in (N \cup \Sigma)^*$ be strings of terminals and nonterminals. β is *derivable in one step from* α if we can pick some nonterminal A in α , apply a production $A \rightarrow \gamma$, and get β .

Notation: $\alpha \xrightarrow[G]{1} \beta$

Ex: $aaaaSbbbb \xrightarrow[G]{1} aaaaaSbbbb$

What else?

Even more formally: $\alpha \xrightarrow[G]{1} \beta$ if

- $\alpha = \alpha_1 A \alpha_2$ for $\alpha_1, \alpha_2 \in (N \cup \Sigma)^*$, $A \in N$
- $A \rightarrow \gamma \in P$
- $\alpha_1 \gamma \alpha_2 = \beta$

$\alpha \xrightarrow[G]{n+1} \beta$ if there exists some $\gamma \in (N \cup \Sigma)^*$ such that $\alpha \xrightarrow[G]{n} \gamma$ and $\gamma \xrightarrow[G]{1} \beta$.

$\alpha \xrightarrow[G]{*} \beta$ if $\alpha \xrightarrow[G]{n} \beta$ for some $n \geq 1$.

The language of a CFG is the set of strings of *terminals* you can derive from S using its productions. Formally:

$$L(G) = \{x \in \Sigma^* \mid S \xrightarrow[G]{*} x\}$$